# ReDCIM: Reconfigurable Digital Computing-In-Memory Processor With Unified FP/INT Pipeline for Cloud AI Acceleration

Fengbin Tu, *Member, IEEE*, Yiqi Wang, Zihan Wu, Ling Liang, Yufei Ding, Bongjin Kim, *Senior Member, IEEE*, Leibo Liu, *Senior Member, IEEE*, Shaojun Wei, *Fellow, IEEE*, Yuan Xie, *Fellow, IEEE*, and Shouyi Yin, *Member, IEEE*

***Abstract*— Cloud AI acceleration has drawn great attention in recent years, as big models are becoming a popular trend in deep learning. Cloud AI runs high-efficiency inference, high-accuracy inference and training, in demand of flexible floating-point (FP)/integer (INT) multiply–accumulation (MAC) support. Many computing-in-memory (CIM) processors have been proposed for efficient AI acceleration. They usually rely on analog CIM techniques that are only suitable for high-efficiency neural network (NN) inference with low-precision INT MAC support. Since cloud AI demands high efficiency, high accuracy, and high flexibility simultaneously, we propose an innovative architecture reconfigurable digital CIM (ReDCIM) that meets all three requirements. We design the first CIM-based cloud AI processor, ReDCIM, which constructs a unified FP/INT pipeline architecture based on exponent pre-alignment and reconfigurable in-memory accumulation. Bitwise in-memory Booth multiplication is proposed to reduce computation on CIM. The fabricated ReDCIM chip achieves a state-of-the-art energy efficiency of 29.2 TFLOPS/W at BF16 and 36.5 TOPS/W at INT8.**

***Index Terms*— Booth multiplication, computing-in-memory (CIM), floating-point (FP), neural network (NN), reconfigurable architecture.**

Manuscript received 7 May 2022; revised 1 September 2022 and 7 November 2022; accepted 8 November 2022. Date of publication 2 December 2022; date of current version 28 December 2022. This article was approved by Associate Editor Sophia Shao. This work was supported in part by the National Key Research and Development Program under Grant 2018YFB2202600, in part by NSFC under Grant U19B2041 and Grant 61774094, in part by the Beijing S&T Project under Grant Z191100007519016, and in part by the Beijing Innovation Center for Future Chip. *(Corresponding author: Shouyi Yin.)*

Fengbin Tu is with the School of Integrated Circuits, the Beijing Innovation Center for Future Chip, and the Beijing National Research Center for Information Science and Technology, Tsinghua University, Beijing 100084, China, and also with the Department of Electrical and Computer Engineering, University of California at Santa Barbara, Santa Barbara, CA 93106 USA.

Yiqi Wang, Zihan Wu, Leibo Liu, Shaojun Wei, and Shouyi Yin are with the School of Integrated Circuits, the Beijing Innovation Center for Future Chip, and the Beijing National Research Center for Information Science and Technology, Tsinghua University, Beijing 100084, China (e-mail: yinsy@tsinghua.edu.cn).

Ling Liang, Bongjin Kim, and Yuan Xie are with the Department of Electrical and Computer Engineering, University of California at Santa Barbara, Santa Barbara, CA 93106 USA.

Yufei Ding is with the Department of Computer Science, University of California at Santa Barbara, Santa Barbara, CA 93106 USA.

Color versions of one or more figures in this article are available at https://doi.org/10.1109/JSSC.2022.3222059.

Digital Object Identifier 10.1109/JSSC.2022.3222059

## I. Introduction

DUE to the increasing size and computation of neural network (NN) models, conventional digital artificial intelligence (AI) processors usually suffer from massive data movements between separate compute and memory. Computing-in-memory (CIM) eliminates the bottleneck by integrating multiply–accumulation (MAC) into memory, such as static random access memory (SRAM) [13], [14], [15], [24], [25] and resistive RAM (ReRAM) [6], [21], [22], which has proved to be a promising architecture for energy-efficient NN acceleration.

Most previous CIM processors are based on analog CIM techniques. They can realize efficient integer (INT) MAC computation, making them quite suitable for high-efficiency NN inference in edge AI tasks. However, analog CIM has two inherent drawbacks in accuracy and flexibility that limit its application to the growing cloud AI scenarios:

1) Analog CIM's non-ideal issues, such as signal margin and process variation, cause accuracy loss. The problem will get severe in high-accuracy scenarios.
2) The analog data path in memory limits the CIM function to be only INT MAC.

Cloud AI acceleration draws great attention in these years since more and more large-scale NN models are developed and run on cloud platforms. As illustrated in Fig. 1, besides high-efficiency inference, cloud AI also needs to run high-accuracy inference and training. This raises additional accuracy and flexibility requirements for cloud AI acceleration:

1) highly accurate support for INT MAC and floating-point (FP) MAC [1], [3];
2) flexible support for different INT and FP MAC operations to satisfy diverse cloud inference and training tasks.

Therefore, cloud AI demands high efficiency, high accuracy, and high flexibility simultaneously. In consideration of the previously mentioned two drawbacks in accuracy and flexibility, analog CIM is not suitable for this scenario.

To meet all the requirements of cloud AI, we propose a new architecture called reconfigurable digital CIM (ReDCIM), which fuses the spirits of digital CIM and reconfigurable computing. Digital CIM directly integrates digital INT MAC into SRAM cells [2], [8], which totally avoids the non-ideal issues of analog CIM while maintaining high

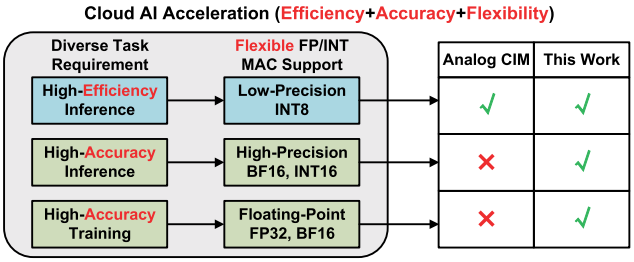



Fig. 1. Cloud AI runs high-efficiency inference, high-accuracy inference, and training, with flexible FP/INT MAC support. Analog CIM is only suitable for high-efficiency NN inference at low-precision INT precision. This work proposes ReDCIM that meets all the three requirements of cloud AI acceleration.

energy efficiency. Reconfigurable computing modifies CIM's fixed INT MAC as reconfigurable FP/INT MAC to strengthen functional flexibility.

In this article, we will demonstrate how to use the proposed architecture to develop AI processors for cloud AI acceleration. Specifically, we find out that designing a CIM processor for cloud AI has three technical limitations (see Fig. 2):

1) FP MAC has tightly coupled exponent alignment and INT mantissa MAC operations. Implementing complex exponent alignment in memory will harm CIM's direct accumulation structure and reduce efficiency.
2) FP MAC's energy is dominated by INT mantissa MAC. Further acceleration on CIM-based INT MAC is critical for the entire processor's energy efficiency.
3) Previous cloud AI processors usually have separate FP and INT engines, but only activate one engine at once [1], [3], causing high area overhead and low resource utilization.

Based on the proposed architecture, we design the first ReDCIM (pronounced as "Red-CIM") processor for cloud AI scenarios, with efficient, accurate, and flexible FP/INT support. ReDCIM has three architecture features from top to bottom to tackle the above limitations.

1) ReDCIM is designed on an in-memory alignment-free FP MAC pipeline that interleaves exponent alignment and INT mantissa MAC. Both FP inputs and weights are pre-aligned to their local maximum exponents, so the CIM focuses on only MAC acceleration with no need for complex alignment logic.
2) The pipeline's CIM-based INT MAC is further optimized with a Bitwise in-Memory Booth Multiplication ($BM^2$) architecture. The $BM^2$ encoders perform bitwise input Booth encoding. The CIM macro is modified with digital logic for partial product recoding. Compared with conventional bit-serial CIM, $BM^2$ reduces nearly 50% cycle count and bitwise multiplications.
3) The proposed pipeline offers great opportunities to reuse CIM for constructing a unified FP/INT MAC dataflow. ReDCIM implements hierarchical and reconfigurable in-memory accumulators to enable flexible support of BF16 (BFloat16)/FP32 and INT8/INT16 in one CIM macro.

This article is the journal extension version of our International Solid-State Circuits Conference (ISSCC) 2022 work [18], providing higher level insights for readers with many additional materials. The most important extension is proposing a new AI processor architecture ReDCIM, overcoming the efficiency, accuracy, or flexibility limitations of previous architectures (digital architecture, analog CIM, and digital CIM). We use our ReDCIM chip presented in ISSCC 2022 to demonstrate how to design a cloud AI processor based on the proposed architecture. Based on the fusion of digital CIM and reconfigurable computing, ReDCIM exploits top-to-bottom techniques to meet the efficiency, accuracy, and flexibility requirements of cloud AI scenarios.

The rest of this article is organized as follows. Section II provides background and motivation. Section III proposes ReDCIM's overall architecture. The three architecture features are described in Sections IV–VI, respectively. Section VII presents experimental results on the 28-nm fabricated ReDCIM chip. Section VIII concludes this article.

## II. Background and Motivation

### A. Analog CIM

As shown in Fig. 3(a), conventional digital AI processors have discrete MAC units and memory. With the increasing size and computation of NN models, there exist frequent data movements between the MAC units and memory, which significantly affects the overall energy efficiency.

Fig. 3(b) shows a typical analog CIM architecture, one mainstream technique that eliminates the above memory access bottleneck by implementing analog MAC in memory, such as SRAM [13], [14], [15], [25] and ReRAM [6], [21], [22]. Usually, input digital-to-analog converters (DACs) and output analog-to-digital converters (ADCs) are designed in the peripheral circuits. For one NN layer ($O = I * W$), inputs are first converted to analog signals and then perform bitwise analog MAC operations with weights ($W$) through current or voltage. The outputs are converted back to digital in the end.

Although analog CIM saves lots of data movements for higher energy efficiency, the two inherent drawbacks limit analog CIM's accuracy and flexibility: 1) analog computing's non-ideal issues cause accuracy loss and 2) the analog data path in memory is difficult to modify, so the CIM function is usually limited to only INT MAC. As demonstrated in Fig. 1, cloud AI demands high efficiency, high accuracy, and high flexibility simultaneously. Therefore, analog CIM is more suitable for low-power edge AI, rather than cloud AI.

### B. Digital CIM

In recent years, there is an emerging CIM architecture called digital CIM [2], [8], which embeds bitwise digital MAC to SRAM, as shown in Fig. 3(c). One of the pioneering works, Column-MAC [8], designs an XNOR gate and a full adder for each SRAM cell. TSMC's digital CIM in ISSCC'21 [2] simplifies the in-SRAM logic by attaching only one NOR gate to each cell and placing accumulators in the subarray level. Digital CIM combines the benefits of digital architectures and CIM: high efficiency is achieved by integrating compute into memory, while digital in-memory logic guarantees high accuracy by avoiding analog non-ideality. However, current digital CIM can only support INT MAC operations, which cannot meet the flexibility requirement of cloud AI.

### C. Reconfigurable Digital CIM

In this article, we propose a new AI processor architecture called ReDCIM, which applies reconfigurable computing into

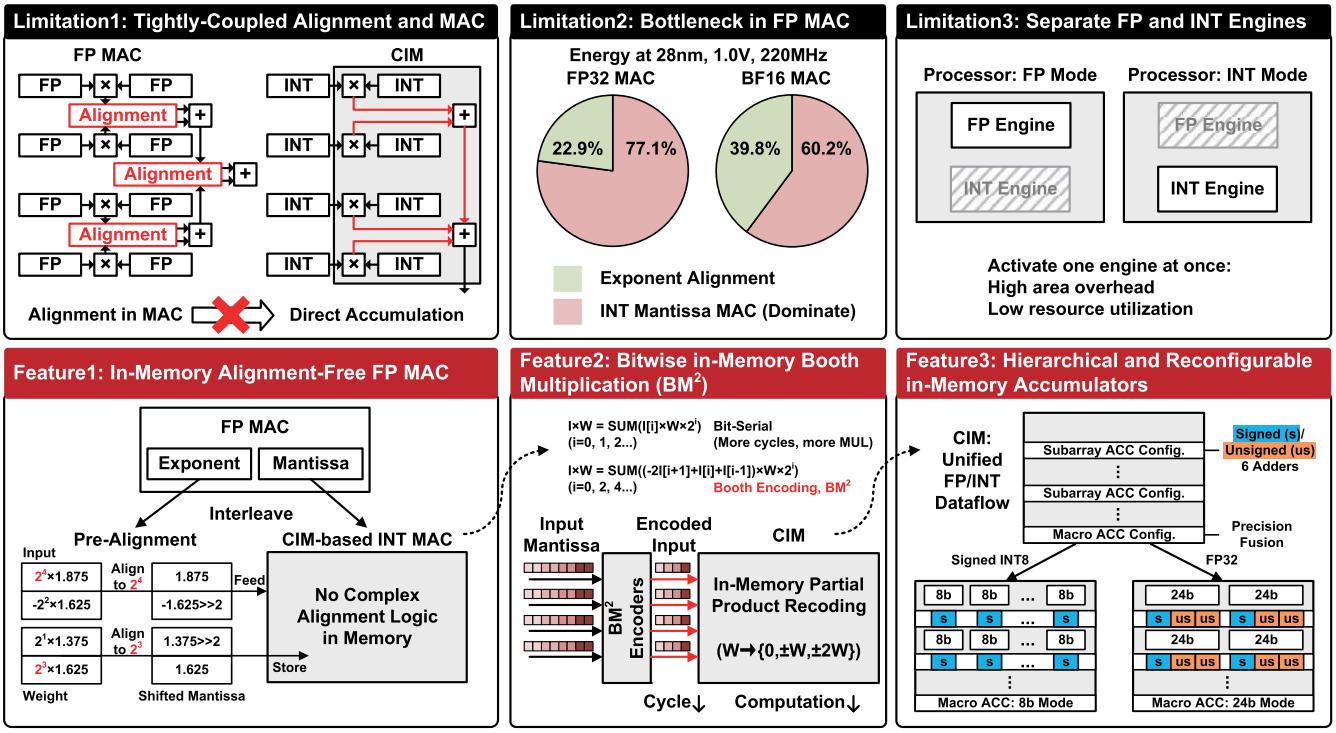



Fig. 2. Technical limitations of designing a CIM processor for cloud AI acceleration and ReDCIM's corresponding features that target the three limitations.

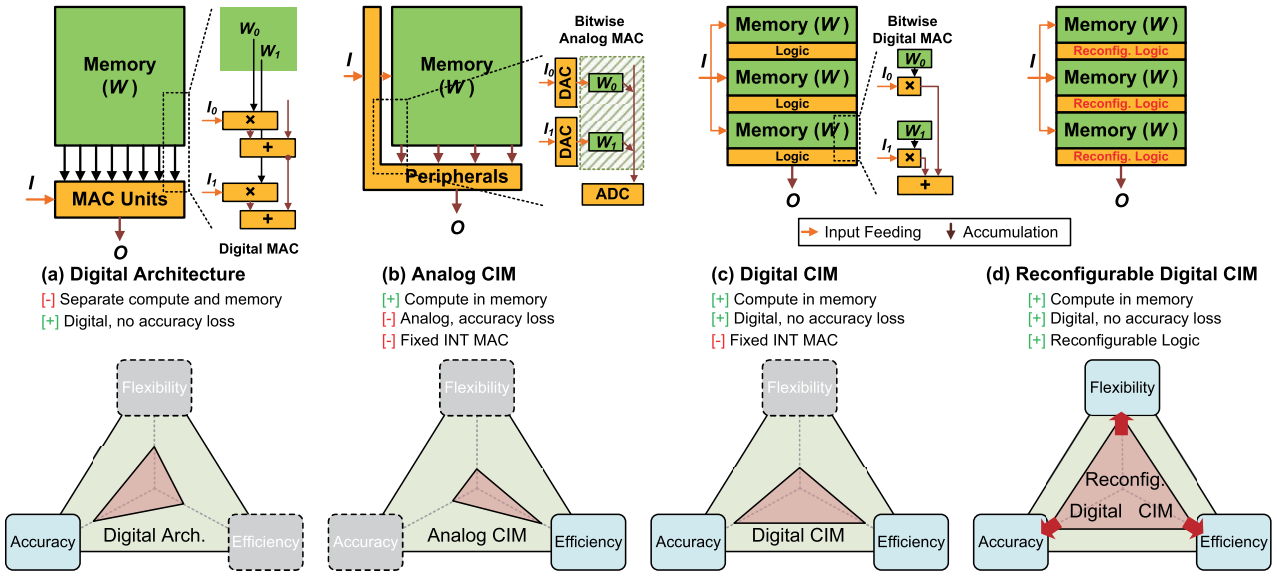



Fig. 3. Different AI processor architectures: (a) digital architecture, (b) analog CIM, (c) digital CIM, and (d) ReDCIM.

digital CIM to meet all the requirements of cloud AI, as illustrated in Fig. 3(d). Reconfigurable computing is a technique that can enhance AI processors' flexibility by dynamically changing the data path for diverse workloads [7], [10], [17], [19], [23]. In the proposed architecture, we use digital CIM to set a foundation for efficient and accurate NN processing, and modify CIM's fixed INT MAC as reconfigurable logic to greatly extend functional flexibility. In the following, we will demonstrate how to design a cloud AI processor based on the proposed architecture.

## III. ReDCIM Processor Architecture

The ReDCIM processor's overall architecture is shown in Fig. 4(a). It consists of 16 CIM cores, a 32-kB global buffer, a single instruction multiple data (SIMD) core, and a top controller. Each core has a 2-kB input-exponent memory (IEMEM), a 4-kB input-mantissa memory (IMMEM), an input alignment unit (IAU), a $BM^2$ controller, and four $BM^2$-CIM macros. Fig. 4(b) illustrates the three features that tackle the limitations pointed out in Fig. 2. With different-level features from top to bottom, ReDCIM presents the fusion of digital

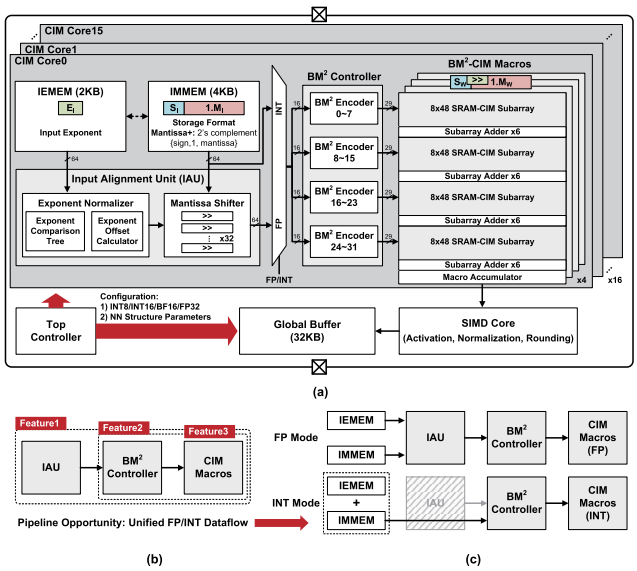



Fig. 4. ReDCIM processor. (a) Overall architecture. (b) Three features from top to bottom. (c) FP and INT mode configurations of the IAU-BM$^2$ controller-CIM macros pipeline in a CIM core.

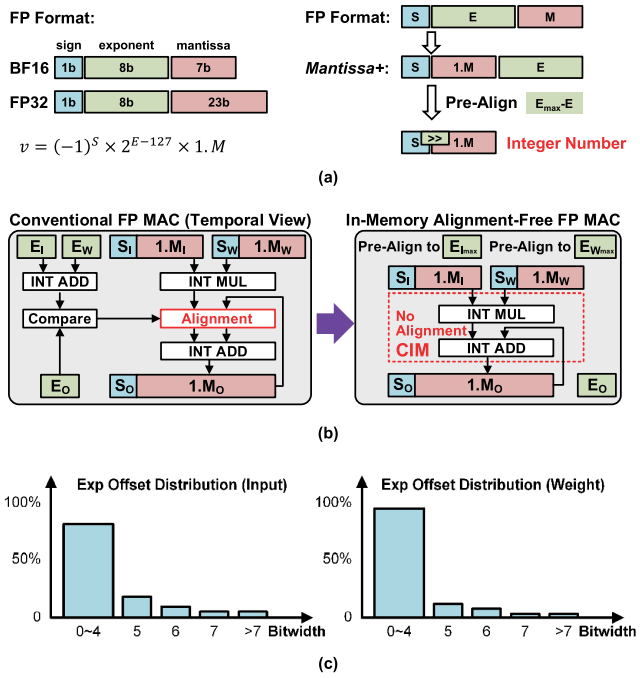



Fig. 5. In-memory alignment-free FP MAC. (a) Standard FP format and *Mantissa+* format. (b) Comparison of conventional FP MAC and in-memory alignment-free FP MAC. (c) Exponent offset distribution of EfficientNet-B0 BF16 inference.

CIM and reconfigurable computing for cloud AI: **Feature1**'s in-memory alignment-free FP MAC pipeline is constructed on IAU-BM$^2$ controller-CIM macros, realizing a unified FP/INT MAC dataflow. **Feature2** relies on the BM$^2$ controller and CIM macros to build the BM$^2$ architecture that enhances the efficiency of digital CIM. **Feature3** implements hierarchical and reconfigurable accumulators in the CIM macro to enable flexible support for different FP and INT precisions.

ReDCIM has two work modes: FP mode for BF16/FP32 precision and INT mode for INT8/INT16 precision. The two modes reuse the same datapath in one CIM core to save hardware resources, as shown in Fig. 4(c). In the FP mode, IEMEM stores input exponent bits, and IMMEM stores input mantissa bits in a new format called *Mantissa+*, which is the 2's complement representation of {sign, 1, mantissa}. IAU conducts exponent pre-alignment for inputs and sends them to the BM$^2$ controller for Booth encoding. The encoded inputs are fed to the BM$^2$-CIM macros and perform INT mantissa MAC with the weights, which are pre-aligned offline and stored also with the *Mantissa+* format. For data reuse, the four macros in each core store different weights and share the same 32 inputs encoded by the BM$^2$ controller. The outputs are sent to the SIMD core for operations such as activation, normalization, and rounding. In the INT mode, the CIM core's IEMEM and IMMEM are combined to store inputs. IAU is bypassed, while inputs are directly sent to the BM$^2$ controller and CIM macros for INT MAC computation.

## IV. In-Memory Alignment-Free FP MAC Pipeline

### A. Conventional FP MAC Versus Our Method

Fig. 5(a) illustrates the standard FP format and the *Mantissa+* format. The standard FP format has three parts, i.e., one sign bit, exponent bits, and mantissa bits. For instance, BF16 and FP32 both have eight exponent bits. They have seven and 23 mantissa bits, respectively. The value of an FP number can be represented as $v = (-1)^S \times 2^{E-127} \times 1.M$, where $S$, $E$, and $M$ are the value of the sign, exponent, and mantissa bits. In FP MAC computation, a "1" bit is usually added between the sign and mantissa bits to construct a fixed-point number to perform INT mantissa MAC. Thus, in this article, we define a new format called *Mantissa+*, which is the 2's complement representation of {sign, 1, mantissa}.

Fig. 5(b) compares conventional FP MAC and our in-memory alignment-free FP MAC. FP MAC computation comprises exponent alignment and INT mantissa MAC. In conventional FP MAC, the INT mantissa MAC is conducted by multiplying the *Mantissa+* values of the input and weight, and then accumulating the product to the partial sum. Since the exponents of the product ($E_I + E_W - 127$) and partial sum ($E_O$) are different, their exponents should be aligned before each accumulation. Therefore, conventional FP MAC has tightly coupled exponent alignment and INT mantissa MAC operations. As illustrated in **Limitation1** of Fig. 2, previous CIM is usually designed only for INT MAC with a direct accumulation structure. Implementing exponent alignment in memory will produce complex logic between every two rows of a CIM macro, which harms the CIM structure and causes high in-memory circuit overhead.

The key idea of our method is interleaving the tightly coupled exponent alignment and INT mantissa MAC to totally avoid alignment in CIM. In our method, we process a group of FP MACs simultaneously: pre-align their input and weight exponents to their local maximum $E_{I\max}$ and $E_{W\max}$. In this way, the exponents of the product ($E_{I\max} + E_{W\max} - 127$) and partial sum ($E_O$) are always the same in this group, so there is no need to perform exponent alignment during INT mantissa MAC, which is friendly to CIM implementation.

### B. In-Memory Alignment-Free FP MAC Pipeline Design

Fig. 6(a) demonstrates the in-memory alignment-free FP MAC pipeline in a CIM core. The key component for input

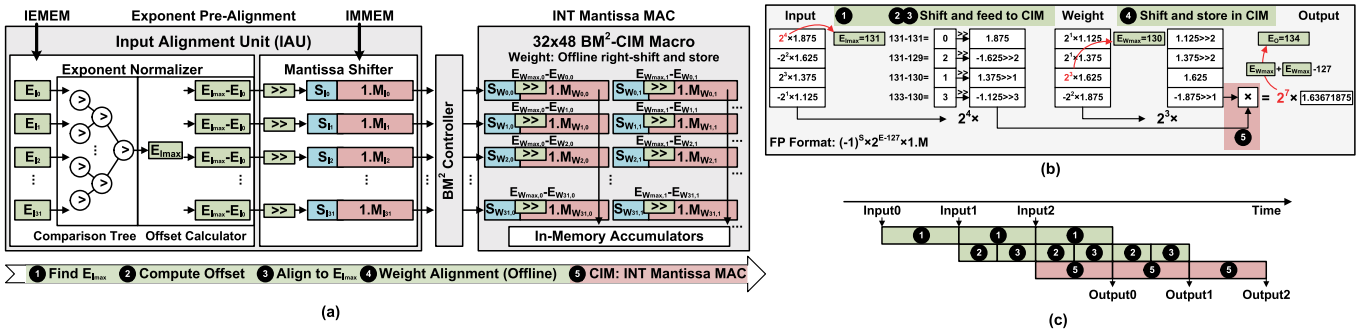



Fig. 6. In-memory alignment-free FP MAC pipeline. (a) IAU-BM$^2$ controller-CIM macros pipeline in a CIM core. (b) Example of four-input and four-weight FP MAC with exponent pre-alignment. (c) Pipeline illustration for our method with a three-group inputs' example.

exponent pre-alignment is the CIM core's IAU, which is composed of an exponent normalizer and mantissa shifter. The pipeline works as follows: ❶ IAU's exponent normalizer loads the exponents of 32 inputs from IEMEM and finds their maximum value $E_{I\max}$ with a comparison tree; ❷ each input's exponent offset is computed by $E_{I\max} - E_I$; ❸ IAU's mantissa shifter loads the corresponding mantissas from IMMEM, and right-shifts them by the offset bits; ❹ as for weights, only the mantissas are stored in CIM. $E_{W\max}$ of each weight column is previously obtained from the NN model; the weight mantissas are stored with our *Mantissa+* format after right-shifting by $E_{W\max} - E_W$ bits; and ❺ after exponent pre-alignment to $E_{I\max}$ and $E_{W\max}$, the shifted input and weight mantissas can directly perform INT MAC operations in CIM. As shown in the simple example of Fig. 6(b), the output exponent equals $E_{I\max} + E_{W\max} - 127$. The final outputs are then transformed into the standard FP format in the SIMD core. Since we pre-align the exponents for every 32 inputs or weights, the values have similar exponents (strong locality) that make the exponent offset usually no more than 4 bits, leading to negligible accuracy loss produced by shifting. We profile the exponent offset distribution for every 32 inputs and weights of EfficientNet-B0. Fig. 5(c) illustrates the distribution histograms of exponent offsets to $E_{\max}$ in every 32 inputs and weights of EfficientNet-B0 inference. 82.9% input exponent offsets and 88.3% weight exponent offsets are 0∼4 bits. The rests are 4 or higher bits, and very few are higher than 7 bits. The large offset indicates a small value with a much lower magnitude than the maximum in 32 data. The extreme case seldom appears and has almost no influence on accuracy.

In the rest of this article, experimental results for FP precision are obtained with the following default settings: 9-b input × 8-b weight for BF16 mantissas and 25-b input × 24-b weight for FP32 mantissas. 9-b input × 8-b weight means that, after exponent pre-alignment, the input and weight *Mantissa+* values are represented in 9- and 8-b numbers. The input has one more bit to retain accuracy. Later, in Section VII-C3, we will show that the default settings achieve negligible accuracy loss on typical benchmarks (see Table I). For NN models that require more bits to preserve high accuracy, ReDCIM can easily extend input bits with more input feeding cycles. The BF16 precision can extend weight bits to 16 and 24 b by configuring the in-memory accumulators.

The proposed FP MAC process works in a pipeline way with three main stages, as illustrated in Fig. 6(c). Stage1 is ❶ finding $E_{I\max}$. Stage2 is ❷ computing input exponent offsets and ❸ aligning input exponents to $E_{I\max}$. Stage3 is ❺ INT mantissa MAC for all the pre-aligned inputs and weights; ❹ weight exponent pre-alignment is conducted offline to save on-chip resources for two reasons: 1) weight information is usually known before computation, so it is possible to pre-process the weights offline and directly store the aligned weights into CIM and 2) even if we can implement weight exponent pre-alignment on the chip, high weight reuse decreases the active ratio of weight pre-alignment logic, which is only active during the initial phase of a layer. The latency is negligible, only consuming 0.66% in the EfficientNet-B0 inference experiment.

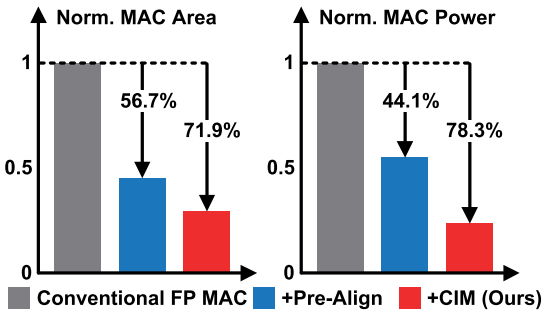



Fig. 7. Evaluation for the in-memory alignment-free FP MAC pipeline. Baseline: digital BF16 MAC + separate weight memory. "+Pre-Align": optimize the baseline with exponent pre-alignment. "+CIM" (Ours): replace digital INT MAC with CIM-based MAC. The baseline results are based on post-synthesis simulation, evaluated at 28 nm, 1.0 V, 220 MHz.

### C. Technique Evaluation

Fig. 7 shows the evaluation for the in-memory alignment-free FP MAC pipeline. The area and power include both MAC logic and weight memory. The baseline is a group of 768 (= 32 × 6 × 4) conventional FP MAC units (BF16 MAC, gray bar) + 0.75-kB weight memory, which has the same MAC count and memory capacity as one CIM core of ReDCIM [see Fig. 4(a)]. ReDCIM can store 32 × 6 weights at BF16 in a CIM macro, and every four macros in a core share the same inputs. Therefore, the baseline processes 32 inputs per cycle, and each input is shared by 6 × 4 weights. We optimize the baseline architecture with our exponent pre-alignment technique (blue bar). The exponent alignment before

each accumulation is replaced with exponent pre-alignment for inputs and weights, as illustrated in Fig. 5(b), obtaining 56.7% smaller area and 44.1% lower power over the baseline. Our final design further replaces the digital MAC units with CIM macros (red bar), achieving the total reduction of 71.9% in area and 78.3% in power.

## V. Bitwise In-Memory Booth Multiplication

### A. Bit-Serial Multiplication Versus BM$^2$

FP MAC's energy is dominated by INT mantissa MAC. As presented in **Limitation2** of Fig. 2, we find out that INT mantissa MAC's energy consumption accounts for 77.1% in FP32 MAC and 60.2% in BF16 MAC. We alleviate this energy bottleneck with further acceleration on CIM-based INT MAC.

Previous digital CIM usually performs multiplication in a bit-serial style [2], [8], as illustrated in (1). $I$ and $W$ are the two operands for multiplication, where $I$ can be expressed as the weighted summation of its $n$ bits ($\sum_{i=0}^{n-1} I[i] \times 2^i$). In conventional digital CIM, each bit of $I$ is first multiplied by $W$ stored in memory and then performs weighted summation in the accumulator. It takes $n$ cycles to bit-serially feed an $n$-bit input and finish the multiplication

$$I \times W = \sum_{i=0}^{n-1} (I[i] \times W \times 2^i). \tag{1}$$

We propose BM$^2$ to provide acceleration for the conventional bit-serial multiplication. Radix-4 Booth encoding is conducted on the input $I$, so (1) can be reformatted as

$$I \times W = \sum_{i=0, i+=2}^{n} ((-2I[i+1] + I[i] + I[i-1]) \times W \times 2^i) \tag{2}$$

where $I[-1] = I[n] = I[n+1] = 0$ for zero padding. With this encoding, two new input bits are processed per cycle, so the cycle count for weighted summation is reduced to $\lfloor (n/2) \rfloor$, which means nearly 50% reduction in cycle count and bitwise multiplications.

Besides input encoding, BM$^2$ also requires recoding partial product on the CIM side according to the encoding signals. In bit-serial multiplication, the partial product is $I[i] \times W \in \{0, W\}$. In BM$^2$, the partial product becomes $(-2I[i+1] + I[i] + I[i-1]) \times W$, recoded as $\{0, \pm W, \pm 2W\}$. This transformation should be implemented in CIM macros. Thus, in Section V-B, the BM$^2$ architecture will be discussed in two parts: BM$^2$ controller and BM$^2$-CIM macros.

### B. BM$^2$ Architecture Design

Fig. 8 shows the BM$^2$ architecture from a CIM subarray view with 8-b weights. As mentioned before, every four BM$^2$-CIM macros in a CIM core share one BM$^2$ controller. There are 32 BM$^2$ encoders in a BM$^2$ controller, and the total size of a BM$^2$-CIM macro is 32 × 48. Thus, in the subarray view of Fig. 8, every eight encoders in the BM$^2$ controller are assigned to one 8 × 48 SRAM-CIM subarray in the BM$^2$-CIM macro. In each subarray, all eight rows are computed all at once. Meanwhile, all four subarrays in a CIM macro are also computed in parallel. For clear demonstration,

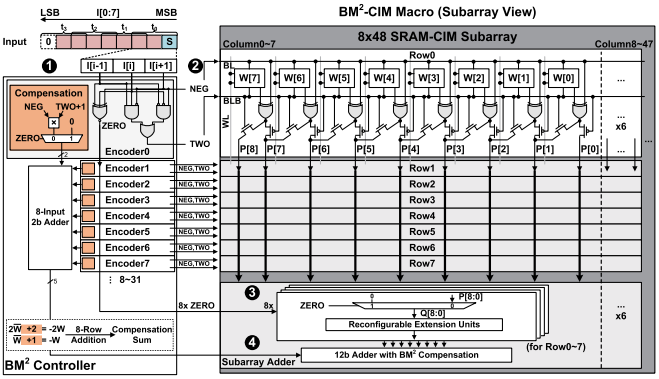



Fig. 8. BM$^2$ architecture: ❶ input Booth encoding; ❷ partial product recoding; ❸ subarray accumulation; and ❹ 2's complement compensation.

| I[i+1] | I[i] | I[i-1] | NEG | TWO | ZERO | P[8:0] | Q[8:0] | Compensation per Row | Partial Product |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 1 | W | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 0 | 0 | W | W | 0 | W |
| 0 | 1 | 0 | 0 | 0 | 0 | W | W | 0 | W |
| 0 | 1 | 1 | 0 | 1 | 0 | 2W | 2W | 0 | 2W |
| 1 | 0 | 0 | 1 | 1 | 0 | $2\overline{W}$ | $2\overline{W}$ | +2 | -2W |
| 1 | 0 | 1 | 1 | 0 | 0 | $\overline{W}$ | $\overline{W}$ | +1 | -W |
| 1 | 1 | 0 | 1 | 0 | 0 | $\overline{W}$ | $\overline{W}$ | +1 | -W |
| 1 | 1 | 1 | 1 | 0 | 1 | $\overline{W}$ | 0 | 0 | 0 |

Fig. 9. BM$^2$ encoder truth table. NEG, TWO, and ZERO are encoding signals. P[8:0] and Q[8:0] are intermediate partial products.

we divide the BM$^2$ architecture into four parts: ❶ input Booth encoding; ❷ partial product recoding; ❸ subarray accumulation; and ❹ 2's complement compensation.

*1) BM$^2$ Controller:* ❶ The BM$^2$ encoder is implemented based on Radix-4 Booth encoding. Each input is fed into a BM$^2$ encoder to generate a set of encoding signals, NEG, TWO, and ZERO per cycle, which recode the partial product as $\{0, \pm W, \pm 2W\}$. The traditional Booth algorithm's signal ONE is replaced by signal ZERO to represent partial product 0 (see the truth table in Fig. 9). This signal ZERO will be later used in the BM$^2$-CIM macro for the zero setting. NEG and TWO are sent to the corresponding row of the subarray through the bitline pair (BL/BLB).

*2) BM$^2$-CIM Macro:* ❷ The BM$^2$-CIM macro is designed with full-digital in-SRAM logic to realize BM$^2$ partial product recoding. A four-transistor XOR gate based on transmission gate logic is connected to the SRAM cell and BL for bitwise negation. Two transistors are added to the XOR output for 1-b left-shifting under the control of signal TWO. Thus, the intermediate partial product P[8:0] can represent $\{W, 2W, \overline{W}, \overline{2W}\}$. ❸ The eight rows of P[8:0] are sent to the subarray adder for zero setting to obtain Q[8:0]. Signal ZERO generated by the BM$^2$ encoder of each row decides whether to directly set P[8:0] as 0. Q[8:0] is then fed to the reconfigurable extension units for sign extension. ❹ Since the values are all in the 2's complement representation, compensation should be further added to guarantee correctness. As highlighted in Fig. 9, $\overline{W} + 1 = -W, 2\overline{W} + 2 = -2W$, so extra 1 and 2 should be compensated for each Q[8:0]. Therefore, every eight encoders in the BM$^2$ controller produce a 5-b compensation sum, which is finally added to eight rows of Q[8:0] in the subarray adder.

We use an ReDCIM architecture instead of an analog CIM, in order to support higher precision (>8 b) with no accuracy loss. Meanwhile, partial product recoding can be

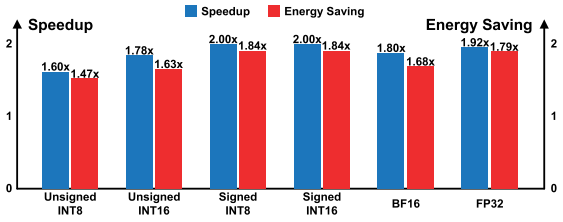



Fig. 10. Evaluation for the BM$^2$ architecture. Baseline: conventional bit-serial digital CIM similar to [2]. BF16 and FP32 include exponent pre-alignment for a fair comparison. The evaluation is based on SPICE-level simulation at 28 nm, 1.0 V, 220 MHz.

easily implemented by customizing digital in-SRAM logic with quite low overhead (only 6.40% in area and 3.06% in power). The reconfigurability enables in-memory shifting, sign extension, and unsigned/signed addition or subtraction, which will be further discussed in Section VI.

### C. Technique Evaluation

Fig. 10 shows the evaluation for the BM$^2$ architecture under unsigned/signed INT8/INT16, BF16, and FP32 precision, based on SPICE-level simulation. The baseline is the conventional bit-serial digital CIM similar to TSMC's design [2]. We implement the baseline in 28-nm CMOS technology and evaluate it at 1.0 V, 220 MHz (same as ReDCIM's highest performance point). The original TSMC digital CIM can only support INT computation [2]. In order to perform evaluations under the FP precision, our baseline extends with additional FP support through the proposed exponent pre-alignment technique. Compared with the baseline, BM$^2$ achieves 1.60×∼2.00× speedup and 1.47×∼1.84× energy saving under different precisions with no input or weight sparsity considered. Although the booth encoding results depend on the 1/0 distribution, the computing cycle count is only related to the input sign and bitwidth, as indicated by (2). Whether an input bit is 1 or 0, it should be encoded by every three sequential bits based on the Radix-4 Booth format, so BM$^2$'s improvement is consistent under different toggle rates. Compared to conventional FP MAC with tightly coupled exponent alignment and INT mantissa MAC, our method brings even higher speedup (2.80× for BF16 and 2.31× for FP32).

## VI. Hierarchical and Reconfigurable In-Memory Accumulators

### A. Unified FP/INT MAC Dataflow

The proposed in-memory alignment-free FP MAC pipeline offers great opportunities to construct a unified FP/INT MAC dataflow, which solves the under-utilization problems in previous cloud AI processors with separate FP and INT engines [1], [3] (**Limitation3**). As illustrated in Fig. 4(c), ReDCIM's FP and INT modes reuse the same IAU-BM$^2$ controller-CIM macros datapath in one CIM core to save hardware resources. In the FP mode, IAU, BM$^2$ controller, and CIM macros are all activated. In the INT mode, IAU is bypassed, and inputs are directly sent to the BM$^2$ controller and CIM macros. Since the CIM macros should be reused by different work modes, we design hierarchical and reconfigurable in-memory accumulators to enable flexible support of BF16/FP32 and INT8/INT16 in one BM$^2$-CIM macro.

### B. In-Memory Accumulator Architecture Design

Fig. 11 demonstrates the hierarchical and reconfigurable in-memory accumulators in a BM$^2$-CIM macro. The 32 × 48 macro is divided into four 8 × 48 subarrays, so each macro has four rows of subarray adders and one macro accumulator. The minimum weight precision in ReDCIM is 8 b, so the macro's 48 columns are divided into six groups. As a result, the **subarray-level** in-memory accumulation has six parallel adders for the corresponding groups. The reconfigurable extension unit controls the sign extension of a subarray adder according to the sign flag and Q[8:7] (the two most significant bits of P[8:0] after the zero setting). The **macro-level** in-memory accumulation has three stages: ❶ six adders that perform vertical accumulation for the four subarrays' partial sums (Psum); ❷ six BM$^2$ accumulators that perform shift accumulation for the bitwise BM$^2$ encoded inputs; and ❸ precision fusion that controls how many columns should merge their sums. The 8-b mode directly sends out all the six sums of BM$^2$ accumulators. The 16-b mode fuses every two sums and generates three outputs. The 24-b mode fuses every three sums and generates two outputs.

The subarray- and macro-level reconfigurations are decided by the operation sign and precision. Fig. 12 presents the detailed configurations for all the supported precision. We will use the two examples in Feature3 of Fig. 2 to explain how the reconfiguration works. For the signed INT8 precision, all the subarray adders are configured as signed 8 b, and only the 8-b fusion mode is enabled. For the FP32 precision, we use 24 b to store the mantissa of an FP32 weight, so every three adders of a subarray are combined to construct an FP32 MAC: The first adder is configured as signed 8 b, and the rest two are configured as unsigned 8 b. At the macro level, the 24-b fusion mode is enabled to merge the results of every 32 columns and generate two outputs per macro.

### C. Technique Evaluation

Fig. 13 shows the evaluation for the unified FP/INT MAC dataflow with hierarchical and reconfigurable in-memory accumulators. The baseline is a digital architecture similar to IBM's design [1] with a separate FP engine for BF16/FP32 and INT engine for INT8/INT16 (blue + gray bar). For a fair comparison, the FP and INT engines have the same MAC count as ReDCIM under the corresponding precision. We optimize the baseline architecture by unifying FP and INT engines with our exponent pre-alignment technique (green bar). Exponent alignment and INT mantissa MAC are interleaved in the FP engine, and then, the INT mantissa MAC units are combined with the INT engine. This optimization obtains 66.1% smaller area and 60.3% lower power. Our ReDCIM further replaces the digital MAC units with reconfigurable CIM macros (red bar), achieving the total reduction of 78.1% in area and 84.8% in power over the baseline. The extra overhead for supporting FP is not very high because we highly reuse the INT logic. IAU only consumes 6.93% area in total, as it is mainly composed of comparing, subtraction, and shifting logic. For the CIM macro, only the macro accumulator's

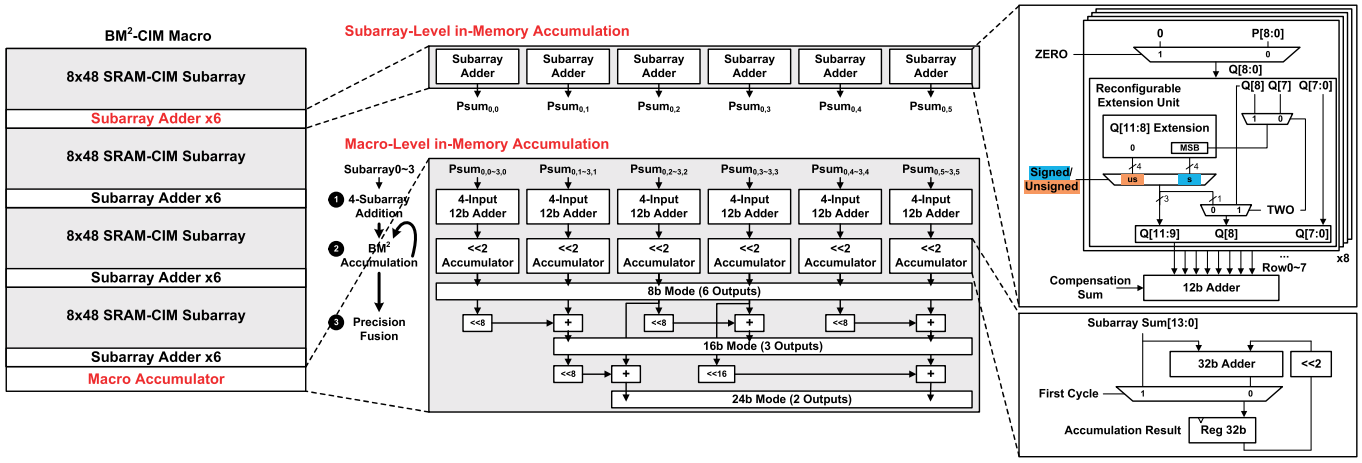
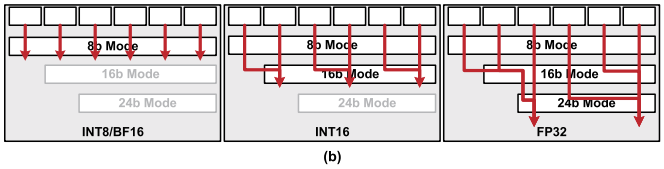



Fig. 11. Hierarchical and reconfigurable in-memory accumulators in a BM$^2$-CIM macro.

| Precision | Sign Extension of 6 Subarray Adders | Subarray Adders per MAC | MACs per Row |
|---|---|---|---|
| INT8/16 (us) | us us us us us us | 1/2 | 6/3 |
| INT8 (s) | s s s s s s | 1 | 6 |
| INT16 (s) | s us s us s us | 2 | 3 |
| BF16 | s s s s s s | 1 | 6 |
| FP32 | s us us s us us | 3 | 2 |

(a)




(b)

Fig. 12. Subarray- and macro-level reconfigurations for all the supported precision (unsigned/signed INT8/INT16, BF16, and FP32). (a) Subarray-level six adder configuration. (b) Macro-level precision fusion configuration.

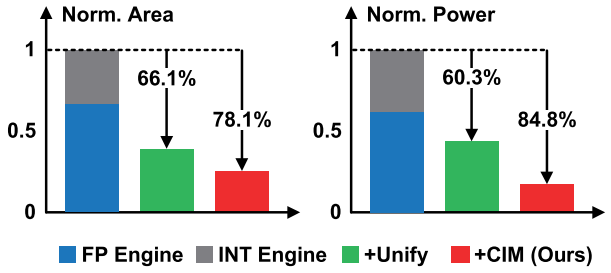
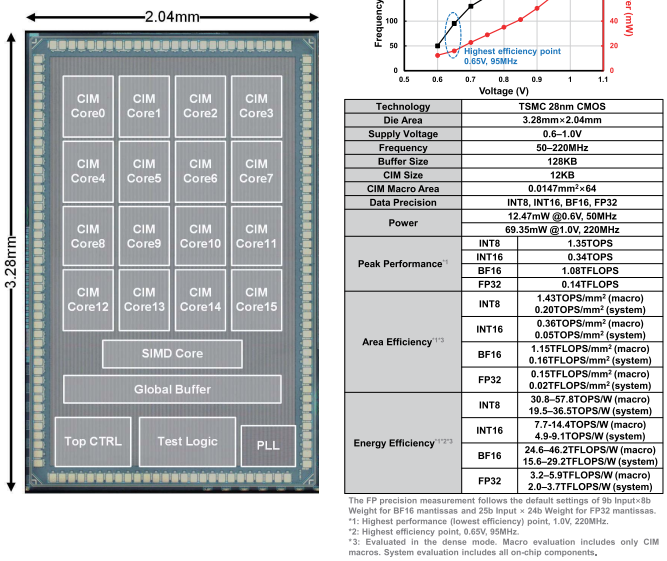



Fig. 13. Evaluation for the unified FP/INT MAC dataflow with hierarchical and reconfigurable in-memory accumulators. Baseline: digital architecture similar to [1], with a separate FP engine for BF16/FP32 and INT engine for INT8/INT16. "+Unify": FP and INT engines are unified with exponent pre-alignment and reconfiguration support. "+CIM" (Ours): BM$^2$ is disabled. All architectures' buffers are excluded for fair comparison on computing engines. The baseline results are based on post-synthesis simulation, evaluated at 28 nm, 1.0 V, 220 MHz.

24-b mode is additionally designed for FP support, which consumes 1.09% area in total.

## VII. Experimental Results

This section first describes ReDCIM's measurement results from a 28-nm fabricated chip. We then provide the test platform setup for evaluation. We conduct comprehensive studies on the ReDCIM processor with cloud AI tasks. Finally, we compare ReDCIM with state-of-the-art works.




| | | |
|---|---|---|
| Technology | TSMC 28nm CMOS | |
| Die Area | 3.28mm×2.04mm | |
| Supply Voltage | 0.6–1.0V | |
| Frequency | 50–220MHz | |
| Buffer Size | 128KB | |
| CIM Size | 12KB | |
| CIM Macro Area | 0.0147mm²×64 | |
| Data Precision | INT8, INT16, BF16, FP32 | |
| Power | 12.47mW @0.6V, 50MHz | |
| | 69.35mW @1.0V, 220MHz | |
| Peak Performance*1 | INT8 | 1.35TOPS |
| | INT16 | 0.34TOPS |
| | BF16 | 1.08TFLOPS |
| | FP32 | 0.14TFLOPS |
| Area Efficiency*1*3 | INT8 | 1.43TOPS/mm² (macro) 0.20TOPS/mm² (system) |
| | INT16 | 0.36TOPS/mm² (macro) 0.05TOPS/mm² (system) |
| | BF16 | 1.15TFLOPS/mm² (macro) 0.16TFLOPS/mm² (system) |
| | FP32 | 0.15TFLOPS/mm² (macro) 0.02TFLOPS/mm² (system) |
| Energy Efficiency*1*2*3 | INT8 | 30.8–57.8TOPS/W (macro) 19.5–36.5TOPS/W (system) |
| | INT16 | 7.7-14.4TOPS/W (macro) 4.9-9.1TOPS/W (system) |
| | BF16 | 24.6–46.2TFLOPS/W (macro) 15.6–29.2TFLOPS/W (system) |
| | FP32 | 3.2–5.9TFLOPS/W (macro) 2.0–3.7TFLOPS/W (system) |

The FP precision measurement follows the default settings of 9b Input×8b Weight for BF16 mantissas and 25b Input × 24b Weight for FP32 mantissas.
*1: Highest performance (lowest efficiency) point, 1.0V, 220MHz.
*2: Highest efficiency point, 0.65V, 95MHz.
*3: Evaluated in the dense mode. Macro evaluation includes only CIM macros. System evaluation includes all on-chip components.

Fig. 14. ReDCIM's chip micrograph and summary.

### A. Chip Micrograph and Summary

Fig. 14 presents the ReDCIM chip micrograph and summarizes the measurement results. ReDCIM is fabricated in 28-nm CMOS technology with 3.28 × 2.04 mm$^2$ die area. The chip can work at 0.6∼1.0-V supply, 50∼220 MHz, with a power consumption of only 12.47∼69.35 mW. ReDCIM supports both FP (BF16/FP32) and integer (INT8/INT16) precision for cloud AI. As illustrated in Fig. 1, cloud AI usually uses INT8/INT16/BF16 for NN inference and BF16/FP32 for NN training. The peak performance reaches 1.35 TOPS for INT8, 0.34 TOPS for INT16, 1.08 TFLOPS for BF16, and 0.14 TFLOPS for FP32 at 1.0 V, 220 MHz. The highest energy efficiency is 36.5 TOPS/W for INT8, 9.1 TOPS/W for INT16, 29.2 TFLOPS/W for BF16, and 3.7 TFLOPS/W

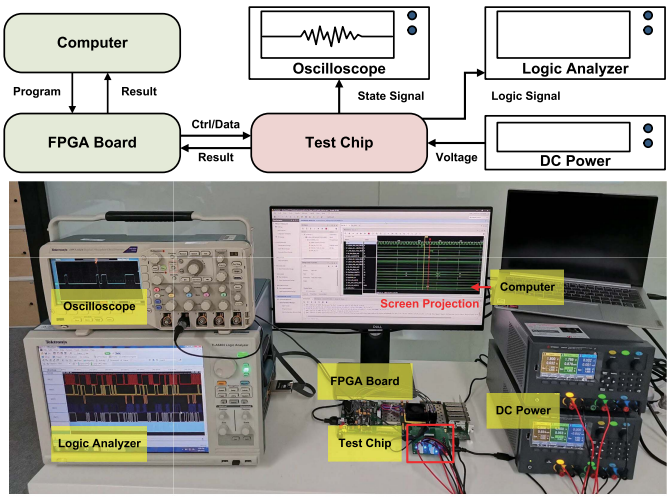



Fig. 15. Test platform for the fabricated ReDCIM processor.

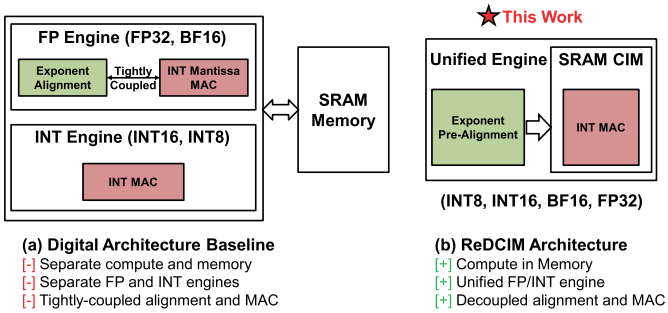
for FP32 at 0.65 V, 95 MHz. The FP precision measurement follows the default settings of 9-b input × 8-b weight for BF16 mantissas and 25-b input × 24-b weight for FP32 mantissas. The INT16 energy efficiency is approximately (1/4) of the INT8 energy efficiency due to the twice bitwidth for both inputs and weights. The BF16 energy efficiency is much higher than the INT16 efficiency because BF16 MAC energy is dominated by INT MAC for the mantissa part, which is 9-b input × 8-b weight in ReDCIM's default setting. The exponent alignment overhead is reduced by the pre-alignment technique and inter-macro input sharing.

### B. Test Platform Setup

Fig. 15 illustrates the test platform for the fabricated ReDCIM processor. The system is composed of a ReDCIM test chip, Xilinx VC709 FPGA board, dc power, logic analyzer, oscilloscope, and host computer. The test dataset and NN model are stored in the FPGA board's DDR3 memory. The FPGA sends control signals and data to the ReDCIM test chip board via the FPGA mezzanine connector (FMC). The dc power supplies 0.6∼1.0-V core voltage for the test chip. The oscilloscope and logic analyzer observe the chip's state signals and logic signals. The computer uses Vivado 2018.2 to generate bitstream and program it into the FPGA board to set up the test platform. As annotated in the photograph, the computer screen is projected onto a monitor. During computation, the test chip sends computing results to the FPGA. The computer receives the results transferred from the FPGA and probes FPGA's internal key signals. The screen displays the results and signals in Vivado, which can be used for further analysis.

### C. Evaluation on Cloud AI Tasks

*1) Digital Architecture Baseline:* To fairly evaluate the benefits of the proposed techniques in ReDCIM, we implement a 28-nm baseline with a digital architecture similar to IBM's ISSCC'21-9.1, a state-of-the-art cloud AI processor [1]. As illustrated in Fig. 16(a), the baseline architecture has separate compute and memory, with a separate FP engine for BF16/FP32 and INT engine for INT8/INT16. The FP engine




**(a) Digital Architecture Baseline**
[-] Separate compute and memory
[-] Separate FP and INT engines
[-] Tightly-coupled alignment and MAC

**(b) ReDCIM Architecture**
[+] Compute in Memory
[+] Unified FP/INT engine
[+] Decoupled alignment and MAC

Fig. 16. Comparison between (a) digital architecture baseline and (b) ReDCIM architecture.

TABLE I
OVERALL EVALUATION ON DIFFERENT CLOUD AI TASKS[*1]

| Model | ResNet-50 | EfficientNet-B0 | EfficientNet-B0 |
|---|---|---|---|
| Dataset | ImageNet | ImageNet | ImageNet |
| Task | Inference | Inference | Training |
| Data Precision | INT8 | BF16 | FP32 |
| Top-1 Accuracy | 75.87% | 76.23% | 76.28% |
| Baseline Accuracy | 76.13% | 76.42% | 76.42% |
| Accuracy Loss[*2] | -0.26% | -0.19% | -0.14% |
| Execution Time | 6.85ms | 0.94ms | 20.8ms[*3] |
| CIM Macro Energy Efficiency | 30.4 TOPS/W | 23.2 TFLOPS/W | 3.0 TFLOPS/W |
| System[*4] Energy Efficiency | 21.7 TOPS/W | 14.0 TFLOPS/W | 1.8 TFLOPS/W |
| Energy Saving[*5] | 11.61x | 8.92x | 9.86x |

[*1]: Measured at 1.0V, 220MHz. Off-chip memory is not included.
[*2]: Compared with the baseline accuracy obtained by FP32 training on GPU.
[*3]: An image's one round of feedforward, backpropagation, and gradient update.
[*4]: Include all on-chip components.
[*5]: The baseline is a 28nm digital architecture with separate FP and INT engines.

uses conventional FP MAC units that have tightly coupled exponent alignment and INT mantissa MAC operations. The FP and INT engines' throughput is normalized to ReDCIM for a fair comparison. Since it is impractical to measure the baseline and all its variants after tapeout, we use post-synthesis simulation to make conservative and reasonable evaluations for our techniques. The baseline architectures are synthesized by the Synopsys design compiler. Power and latency are measured with PrimeTime PX and Synopsys VCS. The energy consumption is usually larger after tapeout due to the additional wiring overhead. We actually make an optimistic estimation for the baseline because the synthesized baseline's energy efficiency is higher than an imagined baseline chip. In this way, we can make sure that our reported benefits are reasonable without exaggeration.

*2) Benchmarks:* We select the following benchmarks for evaluation: ResNet-50 [5] inference (INT8), EfficientNet-B0 inference (BF16), and EfficientNet-B0 [16] training (FP32) on the ImageNet dataset [4]. They represent the three typical cloud AI tasks in Fig. 1: high-efficiency inference, high-accuracy inference, and high-accuracy training, respectively. The NN models are all transformed to the matrix multiplication format through the *im2col* function. The entire inference and training operations are performed on the chip with a proper hardware-friendly approximation for several non-linear functions. For training, pooling/upsampling operations are performed on the SIMD core. Batch normalization is approximated with linear normalization and little accuracy loss.

(a)

| Layer | Description | Latency (μs) | Utilization | Energy (μJ) | Energy Eff. (TOPS/W) |
|---|---|---|---|---|---|
| 0 | Conv1 | 427.64 | 81.67% | 22.74 | 20.76 |
| 1 | Res2a | 370.62 | 92.31% | 21.44 | 21.57 |
| 2 | Res2b | 356.36 | 90.67% | 20.36 | 21.45 |
| 3 | Res2c | 356.36 | 90.67% | 20.36 | 21.45 |
| 4 | Res3a | 669.96 | 93.62% | 39.14 | 21.66 |
| 5 | Res3b | 356.36 | 90.67% | 20.36 | 21.45 |
| 6 | Res3c | 356.36 | 90.67% | 20.36 | 21.45 |
| 7 | Res3d | 356.36 | 90.67% | 20.36 | 21.45 |
| 8 | Res4a | 637.89 | 98.32% | 38.58 | 21.97 |
| 9 | Res4b | 331.42 | 97.49% | 19.92 | 21.92 |
| 10 | Res4c | 331.42 | 97.49% | 19.92 | 21.92 |
| 11 | Res4d | 331.42 | 97.49% | 19.92 | 21.92 |
| 12 | Res4e | 331.42 | 97.49% | 19.92 | 21.92 |
| 13 | Res4f | 331.42 | 97.49% | 19.92 | 21.92 |
| 14 | Res5a | 637.89 | 98.32% | 38.58 | 21.97 |
| 15 | Res5b | 331.42 | 97.49% | 19.92 | 21.92 |
| 16 | Res5c | 331.42 | 97.49% | 19.92 | 21.92 |
| 17 | FC | 3.05 | 99.21% | 0.19 | 22.03 |
| 0~17 | Total | 6.8ms/image | 94.22% | 401.9μJ/image | 21.70 |

(b)

| Layer | Description | Latency (μs) | Utilization | Energy (μJ) | Energy Eff. (TFLOPS/W) |
|---|---|---|---|---|---|
| 0 | Conv3x3 | 35.64 | 56.25% | 1.79 | 12.13 |
| 1 | MBConv1,k3x3 | 53.50 | 34.69% | 2.18 | 9.22 |
| 2 | MBConv6,k3x3 | 102.50 | 52.70% | 4.98 | 11.73 |
| 3 | MBConv6,k3x3 | 75.77 | 62.86% | 4.02 | 12.81 |
| 4 | MBConv6,k5x5 | 44.59 | 75.43% | 2.61 | 13.93 |
| 5 | MBConv6,k5x5 | 51.32 | 71.31% | 2.91 | 13.58 |
| 6 | MBConv6,k3x3 | 34.11 | 63.66% | 1.82 | 12.89 |
| 7 | MBConv6,k3x3 | 39.20 | 75.55% | 2.30 | 13.93 |
| 8 | MBConv6,k3x3 | 39.20 | 75.55% | 2.30 | 13.93 |
| 9 | MBConv6,k5x5 | 44.66 | 85.03% | 2.80 | 14.64 |
| 10 | MBConv6,k5x5 | 70.52 | 86.60% | 4.48 | 14.75 |
| 11 | MBConv6,k5x5 | 70.52 | 86.60% | 4.48 | 14.75 |
| 12 | MBConv6,k5x5 | 45.70 | 89.53% | 2.96 | 14.95 |
| 13 | MBConv6,k5x5 | 44.66 | 98.36% | 3.07 | 15.50 |
| 14 | MBConv6,k5x5 | 44.66 | 98.36% | 3.07 | 15.50 |
| 15 | MBConv6,k5x5 | 44.66 | 98.36% | 3.07 | 15.50 |
| 16 | MBConv6,k3x3 | 60.41 | 92.07% | 3.98 | 15.11 |
| 17 | Conv1x1 | 38.98 | 95.24% | 2.62 | 15.31 |
| 18 | FC | 2.39 | 99.21% | 0.16 | 15.55 |
| 0~18 | Total | 943.0μs/image | 76.29% | 55.6μJ/image | 13.99 |

(c)

| Layer | Description | Latency (μs) | Utilization | Energy (μJ) | Energy Eff. (TFLOPS/W) |
|---|---|---|---|---|---|
| 0 | Conv3x3 | 555.93 | 84.37% | 34.75 | 1.87 |
| 1 | MBConv1,k3x3 | 834.25 | 52.06% | 40.31 | 1.49 |
| 2 | MBConv6,k3x3 | 2397.79 | 52.71% | 116.56 | 1.50 |
| 3 | MBConv6,k3x3 | 1773.08 | 62.84% | 94.08 | 1.64 |
| 4 | MBConv6,k5x5 | 1000.00 | 78.67% | 60.01 | 1.82 |
| 5 | MBConv6,k5x5 | 1130.65 | 75.66% | 66.38 | 1.79 |
| 6 | MBConv6,k3x3 | 762.80 | 66.48% | 41.73 | 1.69 |
| 7 | MBConv6,k3x3 | 858.89 | 80.34% | 52.29 | 1.84 |
| 8 | MBConv6,k3x3 | 858.89 | 80.34% | 52.29 | 1.84 |
| 9 | MBConv6,k5x5 | 991.84 | 89.18% | 64.26 | 1.92 |
| 10 | MBConv6,k5x5 | 1575.25 | 90.31% | 102.84 | 1.93 |
| 11 | MBConv6,k5x5 | 1575.25 | 90.31% | 102.84 | 1.93 |
| 12 | MBConv6,k5x5 | 1068.95 | 88.76% | 69.25 | 1.92 |
| 13 | MBConv6,k5x5 | 1045.02 | 96.45% | 71.73 | 1.99 |
| 14 | MBConv6,k5x5 | 1045.02 | 96.45% | 71.73 | 1.99 |
| 15 | MBConv6,k5x5 | 1045.02 | 96.45% | 71.73 | 1.99 |
| 16 | MBConv6,k3x3 | 1357.73 | 94.01% | 91.70 | 1.97 |
| 17 | Conv1x1 | 868.64 | 98.12% | 60.24 | 2.00 |
| 18 | FC | 55.84 | 94.25% | 3.85 | 1.99 |
| 0~18 | Total | 20.8ms/image | 80.26% | 1.27mJ/image | 1.84 |

Fig. 17. Layerwise analysis on (a) ResNet-50 (INT8) inference, (b) EfficientNet-B0 (BF16) inference, and EfficientNet-B0 (FP32) training at 1.0 V, 220 MHz.

*3) Overall Evaluation:* Table I presents the overall evaluation of different cloud AI tasks. All the results are obtained at 1.0 V, 220 MHz for the highest performance. The CIM macro energy efficiency only considers the energy consumption of CIM macros, while the system macro energy efficiency includes ReDCIM's all on-chip components. The chip-FPGA communication latency is not included in the reported latency to focus on evaluation for the proposed chip architecture itself. ReDCIM achieves 21.7 TOPS/W system energy efficiency on ResNet-50 inference, saving 11.61× energy over the digital architecture baseline. ResNet-50 baseline accuracy (76.13%) is obtained with the default training parameters recommended by Pytorch. Owing to the full-digital in-SRAM logic, ReDCIM has only 0.26% accuracy loss on ResNet-50 inference, much lower than the loss reported in a state-of-the-art analog CIM chip [24]. The analog CIM results in 2.24%∼3.14% accuracy loss on the CIFAR10 dataset [9] and 3.45% loss for the ResNet-18 inference task on ImageNet, mainly due to quantization and block-wise sparsity training. The accuracy can get improved with model training optimization techniques, such as LR optimizations, TrivialAugment, long training, and random erasing. Since ReDCIM focuses on efficient FP/INT support in CIM rather than algorithm techniques, we just use the baseline setting for evaluation.

EfficientNet-B0 is a modern compact model that is more sensitive to quantization than ResNet-50, so we use ReDCIM's BF16 mode rather than INT8 mode to maintain accuracy for inference. We train EfficientNet-B0 at FP32 on GPU and get the baseline accuracy of 76.42%, which almost reproduces the 76.3% accuracy reported in the EfficientNet paper [16]. The BF16 inference accuracy loss is only 0.19%. The system energy efficiency is 14.0 TFLOPS/W, 8.92× higher than the digital architecture baseline in the BF16 mode. We use the last ten epochs of GPU baseline training to evaluate the ReDCIM's FP32 mode. ReDCIM's FP32 accuracy loss is only 0.14% in comparison with the baseline accuracy. The system energy efficiency is 1.8TFLOS/W, 9.86× higher than the digital architecture baseline in the FP32 mode. The result shows that the exponent pre-alignment's slight offset in the least significant bits (LSBs) has only negligible accuracy loss for FP32 training. There are mainly two reasons.

1) Strong value locality makes the exponent offset usually no more than 4 bits. 79.8% input exponent offsets and 82.6% weight exponent offsets are 0∼4 bits in the EfficientNet-B0 FP32 training. With the default setting of 25-b input × 24-b weight for FP32 mantissas, the slight offset means that most of the significant bits are preserved.
2) Prior study shows that NN models are more sensitive to the exponent magnitude rather than the mantissa precision [20], so a slight loss in the mantissa LSB does not make a significant influence on accuracy. Besides, ReDCIM also has extension support for cases that require more bits to preserve high accuracy. Input bits can be easily extended with more input feeding cycles.

*4) Layerwise Analysis:* Fig. 17 provides layerwise analysis on ResNet-50 (INT8), EfficientNet-B0 (BF16) inference, and EfficientNet-B0 (FP32) training at 1.0 V, 220 MHz. For ResNet-50, most layers achieve more than 90% CIM utilization (actual performance/peak performance). ResNet-50's first layer "Conv1" obtains relatively lower utilization (81.67%) because the layer only has three input channels. The few channel count leads to limited elements in one dimension of the weight matrix, which produces lower workload parallelism and more waste of CIM utilization. Overall, ResNet-50 inference on ReDCIM achieves CIM utilization of 94.22%, the latency of 6.8 ms/image, the energy of 401.9 $\mu$J/image, and 21.70 TOPS/W INT8 energy efficiency. Since EfficientNet-B0 is a more compact NN model than ResNet-50, the early layers of EfficientNet-B0 have fewer channels and less workload parallelism, making it not easy to take full utilization of all the CIM resources. As the layer gets deeper, the channel count also increases, so the CIM utilization becomes higher (85.03%∼99.21%) for Layer9∼18. EfficientNet-B0 inference on ReDCIM achieves latency of 943.0 $\mu$s/image, the energy of 55.6 $\mu$J/image, and 13.99 TFLOPS/W BF16 energy efficiency. For EfficientNet-B0 training, we report the FP32 computing latency and energy for an image's one round of feedforward, backpropagation, and gradient update. The energy efficiency is 1.84 TFLOPS/W at FP32. The layerwise results are also the sum of three phases. Although the numbers are much larger than those for BF16 inference, the layerwise variance trend is still similar. We notice that training has higher CIM utilization on the early layers, such as Layer0 and Layer1. This is because FP32 consumes almost three times of CIM logic than BF16, leading to fewer FP32 MAC units per CIM row. The reduced

TABLE II
COMPARISON WITH STATE-OF-THE-ART AI PROCESSORS

| | ISSCC'21-9.1 [1] | ISSCC'21-9.3 [12] | ISSCC'21-15.2 [24] | ISSCC'21-16.4 [2] | VLSI'21 [11] | This Work |
|---|---|---|---|---|---|---|
| Processor Architecture | Digital Architecture | Digital Architecture | Analog CIM | Digital CIM | Digital MAC, Exponent-In-Memory | **Reconfigurable Digital CIM** |
| INT Precision | INT2, INT4 | × | INT2/4/6/8 | INT4/8/12/16 | × | ✓ INT8, INT16 |
| FP Precision | HFP8, FP16, FP32 | FP8-SEB | × | × | BF16 | ✓ BF16, FP32 |
| Technology | 7nm | 40nm | 65nm | 22nm | 28nm | 28nm |
| Supply Voltage (V) | 0.55∼0.75 | 0.75∼1.1 | 0.62∼1.0 | 0.72 | 0.76∼1.1 | 0.6∼1.0 |
| Frequency (MHz) | 1000∼1600 | 20∼180 | 25∼100 | 500 | ∼250 | 50∼220 |
| Die Area ($mm^2$) | 36 | 7.29 | 12 | 0.202 (macro) | 5.832 | 6.69 |
| Power (mW) | - | 13.1∼230 | 18.6∼84.1 | - | 1.2∼156.1 | 12.5∼69.4 |
| Peak Performance[*1] (TOPS or TFLOPS) | 102.4 (INT4, INF) | N/A | 0.013 (INT8, INF) | 0.917 (INT8, INF) | N/A | 1.35 (INT8, INF)[*4] |
| | 25.6 (FP8, TRA) | 0.567 (FP8-SEB, TRA) | N/A | N/A | 0.119 (BF16, TRA) | 1.08 (BF16, INF)[*4]<br>0.14 (FP32, TRA)[*4] |
| Energy Efficiency[*1,2] (TOPS/W or TFLOPS/W) | 16.5 (INT4, INF) | N/A | 2.75 (INT8/4, INF) | 17.7 (INT8, INF)[*3] | N/A | 36.5 (INT8, INF)[*5] |
| | 3.5 (FP8, TRA) | 4.81 (FP8-SEB, TRA) | N/A | N/A | 1.43 (BF16, TRA) | 29.2 (BF16, INF)[*5]<br>3.7 (FP32, TRA)[*5] |

[*1]: Evaluated in the dense mode. One operation (OP) represents one multiplication or one addition. INF: Inference. TRA: Training.
[*2]: System energy efficiency includes all on-chip components. Off-chip memory is not included.
[*3]: On-chip buffers are included for fair comparison. Total buffer capacity is 128KB, same as the ReDCIM processor.
[*4]: Measured at the highest performance point, 1.0V, 220MHz.
[*5]: Measured at the highest efficiency point, 0.65V, 95MHz.

hardware parallelism achieves better balancing with the early layers' reduced workload parallelism with less waste of CIM resources.

*5) Technique Breakdown Analysis:* Fig. 18 demonstrates a technique breakdown analysis on the EfficientNet-B0 inference (BF16) benchmark to clearly illustrate how system energy efficiency is improved with ReDCIM's three features. The digital architecture baseline's energy efficiency is 1.6 TFLOPS/W on EfficientNet-B0 inference (gray bar). We optimize the baseline's FP engine with our exponent pre-alignment technique (blue bar) to reduce frequent exponent alignment before accumulation, achieving 2.0 TFLOPS/W energy efficiency. We further optimize the MAC units in FP and INT engines with input Booth encoding (green bar) to accelerate MAC computation, obtaining 3.4 TFLOPS/W energy efficiency. Unifying FP and INT engines reuses the INT MAC units for FP's INT mantissa MAC operations, which saves power and improves energy efficiency to 6.3 TFLOPS/W (orange bar). Our ReDCIM finally replaces digital MAC units with $BM^2$-CIM macros (red bar), reaching 14.0 TFLOPS/W energy efficiency, which is totally 8.92× higher than the baseline.

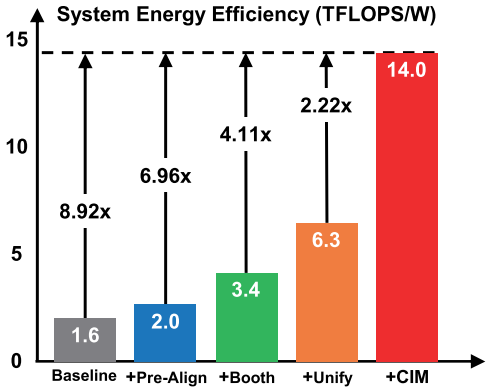



Fig. 18. Technique breakdown analysis on EfficientNet-B0 inference (BF16). "+Pre-Align": optimize the FP engine with exponent pre-alignment. "+Booth": optimize MAC units with input Booth encoding. "+Unify": combine FP and INT engines by reusing the MAC units. "+CIM" (Ours): replace MAC units with $BM^2$-CIM macros. The baseline results are based on post-synthesis simulation, evaluated at 28 nm, 1.0 V, 220 MHz.

## D. Comparison With State-of-the-Art Works

Table II shows a detailed comparison with state-of-the-art AI processors. The reported performance and energy efficiency results are marked with inference (INF) or training (TRA). The architectures of AI processors witness significant development in recent years. Reference [1] is a digital architecture with separate FP and INT engines. Reference [12] is a digital architecture with only FP support. Reference [24] is an analog CIM processor with INT2/4/6/8 support. Reference [2] is a digital CIM processor with integer precision up to INT16 but without FP support. Reference [11] is a hybrid FP architecture that implements exponent alignment in memory and conventional digital units for INT mantissa MAC. Our ReDCIM is the first ReDCIM architecture that supports both FP and INT computation in one unified pipeline. ReDCIM's peak INT energy efficiency is 36.5 TOPS/W at INT8, 0.65 V, 95 MHz, which is 2.06× higher than TSMC's 22-nm INT digital CIM [2], mainly due to the $BM^2$-CIM architecture. ReDCIM's INT8 efficiency is even 2.21× higher than IBM's AI processor [1]'s 16.5 TOPS/W at INT4 and 7 nm. The main reason is that IBM's processor has separate datapaths for INT2/INT4/HFP8/FP16/FP32 multiplication and 8-MB SRAM. The logic other than INT4 computing causes a large power overhead. Owing to the CIM architecture and unified reconfigurable FP/INT datapath (see breakdown benefits in Fig. 18), ReDCIM achieves higher efficiency with INT8 and less advanced technology. ReDCIM's peak FP energy efficiency is 29.2 TFLOPS/W at BF16, 0.65 V, 95 MHz, which is 20.42× higher than KAIST's FP CIM [11]. This is because KAIST's work still relies on digital architecture to build MAC units and designs CIM just for exponent alignment. ReDCIM utilizes CIM to accelerate the more energy-dominated INT mantissa MAC and saves more energy through exponent pre-alignment and $BM^2$.

## VIII. CONCLUSION

AI processors have experienced great architecture development in these years. Conventional digital architecture's energy

efficiency is usually constrained by massive data movements between separate compute and memory. Analog CIM eliminates the bottleneck by integrating analog MAC logic into memory but suffers from accuracy and flexibility limitations. Digital CIM solves the accuracy problem with digital in-memory logic, but the functional flexibility is still limited to INT MAC operations. In this article, we propose an innovative architecture called ReDCIM that balances efficiency, accuracy, and flexibility. Based on the architecture, the first CIM-based cloud AI processor ReDCIM is designed with efficient, accurate, and flexible FP/INT support. The fusion of digital CIM and reconfigurable computing opens up a new direction for AI processor design, with strong scalability for more future AI applications.

## REFERENCES


[1] A. Agrawal et al., "A 7 nm 4-core AI chip with 25.6 TFLOPS hybrid FP8 training, 102.4 TOPS INT4 inference and workload-aware throttling," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 64, Feb. 2021, pp. 144–146.

[2] Y.-D. Chih et al., "An 89 TOPS/W and 16.3 TOPS/mm$^2$ all-digital SRAM-based full-precision compute-in memory macro in 22 nm for machine-learning edge applications," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 64, Feb. 2021, pp. 252–254.

[3] J. Choquette, E. Lee, R. Krashinsky, V. Balan, and B. Khailany, "The A100 datacenter GPU and ampere architecture," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 64, Feb. 2021, pp. 48–50.

[4] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2009, pp. 248–255.

[5] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.

[6] J.-M. Hung et al., "An 8-Mb DC-current-free binary-to-8b precision ReRAM nonvolatile computing-in-memory macro using time-space-readout with 1286.4–21.6TOPS/W for edge-AI devices," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 65, Feb. 2022, pp. 182–184.

[7] S. Kang et al., "GANPU: A 135TFLOPS/W multi-DNN training processor for GANs with speculative dual-sparsity exploitation," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2020, pp. 140–142.

[8] H. Kim, Q. Chen, T. Yoo, T. T.-H. Kim, and B. Kim, "A 1–16b precision reconfigurable digital in-memory computing macro featuring column-MAC architecture and bit-serial computation," in *Proc. IEEE 45th Eur. Solid State Circuits Conf. (ESSCIRC)*, Sep. 2019, pp. 345–348.

[9] A. Krizhevsky et al., "Learning multiple layers of features from tiny images," M.S. thesis, Univ. Toronto, Toronto, ON, Canada, 2009. [Online]. Available: https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf

[10] J. Lee, C. Kim, S. Kang, D. Shin, S. Kim, and H.-J. Yoo, "UNPU: A 50.6TOPS/W unified deep neural network accelerator with 1b-to-16b fully-variable weight bit-precision," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2018, pp. 218–220.

[11] J. Lee et al., "A 13.7 TFLOPS/W floating-point DNN processor using heterogeneous computing architecture with exponent-computing-in-memory," in *Proc. Symp. VLSI Circuits*, 2021, pp. 1–2.

[12] J. Park, S. Lee, and D. Jeon, "A 40 nm 4.81 TFLOPS/W 8b floating-point training processor for non-sparse neural networks using shared exponent bias and 24-way fused multiply-add tree," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 64, Feb. 2021, pp. 148–150.

[13] X. Si et al., "A 28 nm 64 Kb 6T SRAM computing-in-memory macro with 8b MAC operation for AI edge chips," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2020, pp. 246–248.

[14] J.-W. Su et al., "A 28 nm 64 Kb inference-training two-way transpose multibit 6T SRAM compute-in-memory macro for AI edge chips," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2020, pp. 240–242.

[15] J.-W. Su et al., "A 28 nm 384 kb 6T-SRAM computation-in-memory macro with 8b precision for AI edge chips," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2021, pp. 250–252.

[16] M. Tan and Q. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 6105–6114.

[17] F. Tu, S. Yin, P. Ouyang, S. Tang, L. Liu, and S. Wei, "Deep convolutional neural network architecture with reconfigurable computation patterns," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 25, no. 8, pp. 2220–2233, Aug. 2017.

[18] F. Tu et al., "A 28 nm 29.2 TFLOPS/W BF16 and 36.5 TOPS/W INT8 reconfigurable digital CIM processor with unified FP/INT pipeline and bitwise in-memory booth multiplication for cloud deep learning acceleration," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2022, pp. 254–256.

[19] F. Tu et al., "Evolver: A deep learning processor with on-device quantization–voltage–frequency tuning," *IEEE J. Solid-State Circuits*, vol. 56, no. 2, pp. 658–673, Feb. 2021.

[20] S. Wang and P. Kanwar. (2019). BFloat16: The secret to high performance on cloud TPUs. Google Cloud Blog. [Online]. Available: https://cloud.google.com/blog/products/ai-machine-learning/bfloat16-the-secret-to-high-performance-on-cloud-tpus

[21] C.-X. Xue et al., "A 22 nm 2Mb ReRAM compute-in-memory macro with 121–28 TOPS/W for multibit MAC computing for tiny AI edge devices," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2020, pp. 244–246.

[22] C.-X. Xue et al., "A 22 nm 4 Mb 8b-precision ReRAM computing-in-memory macro with 11.91 to 195.7 TOPS/W for tiny AI edge devices," in *Proc. IEEE Int. Solid- State Circuits Conf. (ISSCC)*, vol. 64, Feb. 2021, pp. 245–247.

[23] S. Yin et al., "A high energy efficient reconfigurable hybrid neural network processor for deep learning applications," *IEEE J. Solid-State Circuits*, vol. 53, no. 4, pp. 968–982, Apr. 2018.

[24] J. Yue et al., "A 2.75-to-75.9 TOPS/W computing-in-memory NN processor supporting set-associate block-wise zero skipping and ping-pong CIM with simultaneous computation and weight updating," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 64, Feb. 2021, pp. 238–240.

[25] J. Yue et al., "A 65 nm computing-in-memory-based CNN processor with 2.9-to-35.8 TOPS/W system energy efficiency using dynamic-sparsity performance-scaling architecture and energy-efficient inter/intra-macro data reuse," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2020, pp. 234–236.


**Fengbin Tu** (Member, IEEE) received the B.S. degree from the School of Electronic Engineering, Beijing University of Posts and Telecommunications, Beijing, China, in 2013, and the Ph.D. degree from the Institute of Microelectronics, Tsinghua University, Beijing, in 2019.

He was a Post-Doctoral Researcher with the Scalable Energy-efficient Architecture Lab (SEAL), Department of Electrical and Computer Engineering, University of California at Santa Barbara, Santa Barbara, CA, USA, from 2019 to 2022. He designed the AI chip Thinker. He has published two books: *Architecture Design and Memory Optimization for Neural Network Accelerators* and *Artificial Intelligence Chip Design*. His research works appeared at top conferences and journals on integrated circuits and computer architecture, including International Solid-State Circuits Conference (ISSCC), Journal of Solid-State Circuit (JSSC), and the International Symposium on Computer Architecture (ISCA). His research interests include computer architecture, reconfigurable computing, in-memory computing, and deep learning.

Dr. Tu won the 2017 ISLPED Design Contest Award. His Ph.D. thesis won the Tsinghua Excellent Dissertation Award.

**Yiqi Wang** received the B.S. degree from the School of Integrated Circuits, Tsinghua University, Beijing, China, in 2021, where he is currently pursuing the Ph.D. degree with the School of Integrated Circuits.

His research interests include deep learning, in-memory computing, and computer architecture.

**Zihan Wu** received the B.S. degree from the School of Microelectronics, Shandong University, Jinan, China, in 2020. He is currently pursuing the M.S. degree with the School of Integrated Circuits, Tsinghua University, Beijing, China.

His research interests include in-memory computing, computer architecture, and deep learning.

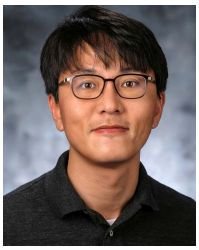
**Ling Liang** received the B.S. degree from the Beijing University of Posts and Telecommunications, Beijing, China, in 2015, and the M.S. degree from the University of Southern California, Los Angeles, CA, USA, in 2017. He is currently pursuing the Ph.D. degree with the Department of Electrical and Computer Engineering, University of California at Santa Barbara, Santa Barbara, CA, USA.

His research interests include tensor computing, neural network security, and spiking neural networks.

**Yufei Ding** received the B.S. degree in physics from the University of Science and Technology of China, Hefei, China, in 2009, the M.S. degree in physics from the College of William and Mary, Williamsburg, VA, USA, in 2011, and the Ph.D. degree in computer science from North Carolina State University, Raleigh, NC, USA, in 2014.

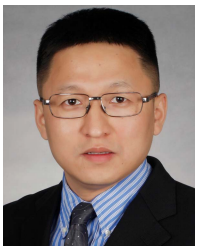
She is currently an Assistant Professor with the Department of Computer Science, University of California at Santa Barbara, Santa Barbara, CA, USA. Her research interests lie in the broad fields of domain-specific language design, architecture and compiler optimization, and hardware acceleration.

Dr. Ding was a recipient of the NSF CAREER Award in 2020, the IEEE Computer Society TCHPC Early Career Researchers Award for Excellence in High Performance Computing in 2019, the NCSU Computer Science Outstanding Dissertation Award in 2018, the NCSU Computer Science Outstanding Research Award in 2016, and the Distinguished Paper Award at Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA) in 2020.



**Bongjin Kim** (Senior Member, IEEE) received the B.S. and M.S. degrees from the Pohang University of Science and Technology (POSTECH), Pohang, South Korea, in 2004 and 2006, respectively, and the Ph.D. degree from the University of Minnesota, Minneapolis, MN, USA, in 2014.

He is currently an Assistant Professor with the Department of Electrical and Computer Engineering, University of California at Santa Barbara, Santa Barbara, CA, USA. His current research interests include quantum-inspired and brain-inspired computing, memory-centric computing architectures, and mixed-signal circuit design.



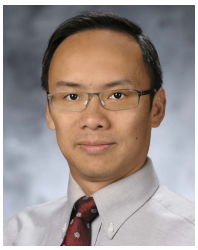
**Leibo Liu** (Senior Member, IEEE) received the B.S. degree in electronic engineering from Tsinghua University, Beijing, China, in 1999, and the Ph.D. degree from the School of Integrated Circuits, Tsinghua University, in 2004.

He is currently a Professor with the Institute of Microelectronics, Tsinghua University. His research interests include reconfigurable computing, mobile computing, and very large scale integration (VLSI) digital signal processing (DSP).

**Shaojun Wei** (Fellow, IEEE) was born in Beijing, China in 1958. He received the Ph.D. degree from the Faculté Polytechnique de Mons, Mons, Belgium, in 1991.

He is currently a Professor with the School of Integrated Circuits, Tsinghua University, Beijing, China. His main research interests include very large scale integration (VLSI) system on chip (SoC) design, electronic design automation (EDA) methodology, and communication application-specific integrated circuit (ASIC) design.

Dr. Wei is a Senior Member of the Chinese Institute of Electronics (CIE).



**Yuan Xie** (Fellow, IEEE) received the B.S. degree in electronic engineering from Tsinghua University, Beijing, China, in 1997, and the M.S. and Ph.D. degrees in electrical engineering from Princeton University, Princeton, NJ, USA, in 1999 and 2002, respectively.

He was an Advisory Engineer with the IBM Microelectronics Division, Burlington, NJ, USA, from 2002 to 2003. He was a Full Professor with Pennsylvania State University, State College, PA, USA, from 2003 to 2014. He was a Visiting Researcher with the Interuniversity Microelectronics Centre (imec), Leuven, Belgium, from 2005 to 2007 and in 2010. He was a Senior Manager and a Principal Researcher with the AMD Research China Lab, Beijing, from 2012 to 2013. He is currently a Professor with the Department of Electrical and Computer Engineering, University of California at Santa Barbara, Santa Barbara, CA, USA. His interests include very large scale integration (VLSI) design, electronic design automation (EDA), computer architecture, and embedded systems.

Dr. Xie is also an AAAS Fellow and an ACM Fellow. He is an expert in computer architecture and has been inducted into the International Symposium on Computer Architecture (ISCA)/International Symposium on Microarchitecture (MICRO)/International Symposium on High-Performance Computer Architecture (HPCA) Hall of Fame.

**Shouyi Yin** (Member, IEEE) received the B.S., M.S., and Ph.D. degrees in electronic engineering from Tsinghua University, Beijing, China, in 2000, 2002, and 2005, respectively.

He was a Research Associate with Imperial College London, London, U.K. He is currently a Full Professor and the Vice-Director of the School of Integrated Circuits, Tsinghua University. He has published more than 100 journal articles and more than 50 conference papers. His research interests include reconfigurable computing, AI processors, and high-level synthesis.

Dr. Yin has served as a Technical Program Committee Member in the top very large scale integration (VLSI) and electronic design automation (EDA) conferences, such as Asian Solid-State Circuits Conference (A-SSCC), International Symposium on Microarchitecture (MICRO), Design Automation Conference (DAC), International Conference on Computer-Aided Design (ICCAD), and Asia and South Pacific DAC (ASP-DAC). He is an Associate Editor of IEEE Transactions on Circuits and Systems I: Regular Papers (TCAS-I), ACM Transactions on Reconfigurable Technology and Systems (TRETS), and *Integration, the VLSI Journal*.